

# Capability Term Rewriting

Luc Alexander Lüthi

May 16th, 2026

## Contents

### 1 Introduction

Abstract interpretation is taken to mean different semantic interpretations of the same program representation. We can use this to model constraints on programs. This is easiest to imagine if you consider types their own program space, where terms are defined as relations which compose in their own algebra when applied. This same idea can be translated to create constrained representations as alternate program spaces of resource consumption, algorithmic complexity, liveness, and classification.

### 2 Programs and Constraints

Programs are constrain implementations. When you implement a program you are implicitly writing a constriction of the program representation space that accepts your implementation as a source of truth for the task. Types are another constraint, and they operate on the same program text. When you call a typed term with an argument, an algebraic rule fires to compute the new type of that program implementation, just like the value of the program itself is computed, there is really nothing discriminating which representation is the target program and which is the constraint. You could say "compute the sum of this list such that the result type is an integer" or you could say "provide a proof of an integer given an array such that the constraint universe computes the sum of terms".

Resource cost is another easy representation. Suppose instead of applying a type to a term, you applied a cost for its use. Then the cost of a greater term could be the sum of its internal calls. Algorithmic complexity and resource complexity can be "typed" by their parametric cost, where compositions of passes like fold, map, filter, each are given a complexity of  $+n$ , and their inclusion compounds to provide a final complexity which is compared to the outer complexity of the implemented term they are used in. Just like types, a custom comparison and evaluation algebra is required for these constraints to be defined.

Another is liveness analysis. If you open a continuous resource like memory or a file handle, that places the program in a certain state, a terms liveness "type" may ask for the state of certain resources to be closed, where then opening a resource would put the state of that representation algebra in the open state, and therefore would not "typecheck".